



BackPAC API

Every endpoint the backend has, in plain words.

19 endpoints · base path `/api/v1` · FastAPI · written for someone reading this codebase for the first time

If you read one thing, read this

Three different callers knock on this backend, and each has its own key.

WHO KNOCKS	WHAT THEY SEND	WHICH ENDPOINTS
The app	a user login token — <code>Authorization: Bearer ...</code>	<code>/sessions*</code>
The agent	a shared password — <code>X-Service-Token</code>	<code>/sessions/internal/*</code>
Anyone	nothing	search, voice, saved trips, health

The app never talks to the agent directly. It only ever talks to this backend.

— All 19 endpoints at a glance

#	METHOD & PATH	IN ONE LINE	CALLED BY
1	GET <code>/healthz</code>	"Are you alive?"	monitor
2	GET <code>/readyz</code>	"Are you set up properly?"	you
3	POST <code>/api/v1/sessions</code>	Start (or resume) a voice call	app
4	GET <code>/api/v1/sessions</code>	List my past chats	app
5	GET <code>/api/v1/sessions/{id}</code>	One chat, in full	not yet
6	PATCH <code>/api/v1/sessions/{id}</code>	Rename / archive / bookmark	app
7	DELETE <code>/api/v1/sessions/{id}</code>	Delete one chat	app
8	DELETE <code>/api/v1/sessions</code>	Delete all my chats	not yet
9	POST <code>.../sessions/internal/messages</code>	Save one line of the conversation	agent

10	POST .../sessions/internal/title	Name the conversation	agent
11	POST .../sessions/internal/trip-results	Save a trip card that was shown	agent
12	POST /api/v1/trips/search/trains	Find trains	agent
13	POST /api/v1/trips/search/flights	Find flights	agent
14	POST /api/v1/trips/search/stays	Find hotels	agent
15	POST /api/v1/trips/saved	Save a trip	not yet
16	GET /api/v1/trips/saved/{user_ref}	List saved trips	not yet
17	GET /api/v1/voice/greeting	The spoken "hello"	app
18	GET /api/v1/voice/welcome-lines	All the hello lines at once	app
19	GET /api/v1/voice/line.wav	The actual sound file	app

"not yet" means the endpoint works, but no screen calls it today. It is built and waiting.

1 Health — is the server okay?

GET /healthz

DOES	Replies <code>{"status":"ok"}</code> .
IN THE UI	None. You never see it.
HOW	Returns a fixed answer instantly. It checks nothing, <i>on purpose</i> — so a slow database can't make the server look dead.

GET /readyz

DOES	Tells you what <i>is</i> and <i>isn't</i> configured.
IN THE UI	None — this is your debugging tool.
HOW	Reports flags the server already knows, like <code>livekit_configured: false</code> and <code>database_ready: true</code> .

Why you care: when voice calls won't start, open this first. It usually answers the question in one look.

2 Sessions — the heart of the app

A “session” is one conversation with the AI. Everything here needs a **logged-in user**, and you only ever see **your own** rows.

POST /api/v1/sessions ★ THE MOST IMPORTANT ONE

DOES	Starts a voice call and hands back a room to join.
IN THE UI	You tap the mic / “start talking”. This fires.
HOW	Four steps, <i>in this order</i> : 1. Creates a row in the database for the conversation. 2. Asks LiveKit for a room + a token (LiveKit carries the live audio). 3. Tells the agent “go join this room”. 4. Only <i>then</i> saves the row for good, and returns the room + token.

Why that order: if the agent is down, the call was never going to work — so no row is saved. Your history stays clean instead of filling up with chats that never happened.

Resuming: send `resumeSessionId` and it reuses the *old* room name. The agent remembers the earlier conversation, and you get the last 50 messages back to paint on screen.

GET /api/v1/sessions

- DOES** Your conversations, newest first.
- IN THE UI** The **History** screen, and the home-screen list.
- HOW** Reads your rows from the database, a page at a time.
- HANDY** `?limit=20&offset=0` for paging · `?saved=true` for bookmarked only · `?status=archived` for the archive.

GET /api/v1/sessions/{id} *not wired up yet*

- DOES** One conversation, complete — every message and every trip card.
- IN THE UI** Nothing today. The app currently gets recent messages from `POST /sessions` when resuming.
- HOW** One lookup, filtered by your user id so you can't read someone else's.

PATCH /api/v1/sessions/{id}

- DOES** Changes one thing about a chat. Four buttons, one endpoint.
- IN THE UI** The menu on a history item — **Rename**, **Archive**, **Bookmark**.
- HOW** Send only the field you want to change.

SEND THIS	RESULT
<code>{"title": "Goa trip"}</code>	renames it
<code>{"status": "archived"}</code>	hides it away
<code>{"status": "active"}</code>	brings it back
<code>{"saved": true}</code>	bookmarks it

Note: archived and bookmarked are separate. "I'm done with this" and "I want to find this again" are different wishes.

DELETE /api/v1/sessions/{id}

- DOES** Erases one chat — the row, its messages, its cards.
- IN THE UI** **Delete** in the history item menu.
- HOW** A real delete. Gone. Use `PATCH` + archive if you might want it back.
- RETURNS** `204` — success, nothing to send back.

DELETE /api/v1/sessions *not wired up yet*

- DOES** Erases **everything** you ever said here.
- IN THE UI** Nothing today — this is the "delete my account" half.
- HOW** The app would delete your login; this deletes your data. It is also the honest answer when someone asks you to erase their data by law.

3 Internal — the agent writing while you talk

These three are **hidden**. They don't appear on `/docs`, and the app can't use them. Only the agent can, using a shared password.

Two things they all share. They identify the conversation by **room name** — because the room is the only thing the agent knows about it. And they all return `202 Accepted`, meaning *"got it, I'll deal with it."* Not `201 Created`. The difference matters: the agent is mid-phone-call and must not stand around waiting to hear where the message was filed.

POST `/api/v1/sessions/internal/messages`

- DOES** Saves one line of the chat — yours or the agent's.
- IN THE UI** This is *why* your old conversation is still there tomorrow.
- HOW** Finds the session by room name, appends one message row.

POST `/api/v1/sessions/internal/title`

- DOES** Names the conversation.
- IN THE UI** The title on each History card.
- HOW** The agent waits until it understands the conversation, then sends a name like *"Weekend in Goa"*. Better than guessing from the first sentence.

POST `/api/v1/sessions/internal/trip-results`

- DOES** Saves a trip card that appeared on screen.
- IN THE UI** The flight / train / hotel cards — so they're still there when you reopen the chat.
- HOW** Stores the card exactly as it was shown.

4 Trips — searching, and saving

POST `/api/v1/trips/search/trains · /flights · /stays`

DOES Finds trains, flights, or places to stay.

IN THE UI You say “find me a flight to Goa”. You never press a search button — the agent decides to call these while you talk.

HOW No database at all. The request goes straight out to an outside travel provider and comes back.

SEND `{"origin": "DEL", "destination": "GOI", "date": "2026-10-02"}`

What's real today: **Flights** are real, if a Travelpayouts key is configured — sample data if not. **Trains and stays** are sample data, marked `TODO` in the code, waiting for a provider.

Nice detail: if the provider breaks you get an **empty list**, not an error. A failed search should make the agent say “I couldn’t find anything” — not crash your phone call.

POST `/api/v1/trips/saved` *not wired up yet*

DOES Saves a trip you liked.

HOW This is the **teaching example** for the codebase — the one endpoint that walks the full path `router → service → repository → database`. Copy it when you build something new.

RETURNS `201 Created`

GET `/api/v1/trips/saved/{user_ref}` *not wired up yet*

DOES Lists someone’s saved trips.

HOW Reads them back out of the database.

⚠ Careful: the user is in the **URL**, not a login token. Anyone who guesses a `user_ref` can read those trips. Fine for a demo — **must move to a real login before this ships**.

5 Voice — the talking welcome screen

GET `/api/v1/voice/greeting`

DOES Gets the spoken “hello” for the welcome screen.

IN THE UI The glowing orb on first open — it speaks, and pulses in time.

HOW The backend asks the **agent** to make the audio (the ElevenLabs key lives there, not here). Returns the text plus a “loudness track” the orb uses to pulse.

If the agent is down: the app just stays quiet. A hello is nice, not essential.

GET `/api/v1/voice/welcome-lines`

DOES Every line the orb can say, in one go.
IN THE UI Tap the orb, it answers **instantly**.
HOW The app fetches this once at startup and keeps it. No waiting later.

GET `/api/v1/voice/line.wav`

DOES The actual sound file for one line.
IN THE UI The audio you hear.
HOW Returns a real `.wav` file — not text-encoded audio inside JSON — so the browser and the phone can cache it normally.

Cached forever: the same words always make the same sound, so it's marked `immutable` and downloaded only once, ever.

6 What the status codes mean here

CODE	MEANING IN THIS BACKEND
200	Here's your answer.
201	Made a new thing (saved trip).
202	Got it, I'll handle it — the agent doesn't wait.
204	Done. Nothing to send back (deletes).
401	You're not logged in, or your login expired.
404	Not found — or it isn't yours. Deliberately the same answer, so nobody can probe for which ids exist.
503	Something I depend on isn't ready (no database, or LiveKit not configured). Check <code>/readyz</code> .

7 Try it yourself

The backend generates a live, clickable page for every public endpoint:

```
http://localhost:8000/docs
```

You can fire real requests from that page. The hidden `/internal/*` three won't be listed there — that's intentional.

Health checks need no setup at all:

```
curl http://localhost:8000/healthz    # {"status":"ok"}
curl http://localhost:8000/readyz    # what's configured
```

BackPAC API — Beginner's Guide. 19 endpoints, verified against the router definitions in `app/domains/*/router.py` and `app/health.py`. To export as PDF: open this file in a browser and print (⌘P / Ctrl+P) → Save as *PDF*. Keep "Background graphics" switched on.